Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

TeamForge – Skill-Based Hackathon & Project Team Matchmaker

Team Name: *Team #*	*TEAMFORGE* *DSCPP-III-2026-T238*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Fartiyal, Mukul – 2510370466* *mukulfartiyal77@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bisht, Anubhav – 2510370178* *anubhav6875@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Singh, Siddhant – 2510380233* *siddhantsingh1890@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Mehul – 2510370093* *mehul.200702@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

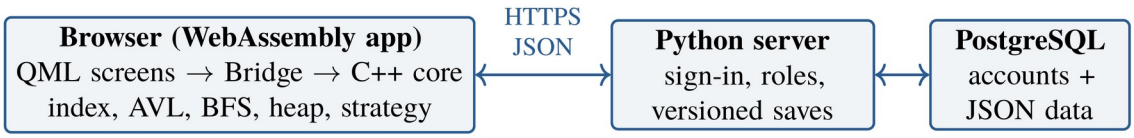
(Brief restatement of your project's main goal. Max 300 words).

- TeamForge helps students form hackathon, PBL and course-project teams based on complementary skills instead of only friend circles.
- A participant adds the skills they can offer (level 1–5) and the skills they want to learn. A host adds a project with its required skills, minimum levels and team size.
- TeamForge finds the students who offer those skills, scores each of them against the team being built and shows a ranked list. For every candidate it shows which skills are covered and which are still missing, so the team can see why someone was suggested.
- In Phase II we built the complete application in C++17 and Qt 6. The core contains the matching engine and the data structures from TCS-302 (hash map, AVL tree, graph with BFS, priority queue). The design uses the OOP concepts from TCS-307 (abstract classes, virtual functions, templates, operator overloading and exceptions).
- The app has three workspaces: Participant, Host and Developer. It comes with 560 sample participants, 8 projects and 549 skills.
- We also made the same app available in the browser. It is compiled to WebAssembly and hosted on Render with sign-in and a shared database: **`https://teamforge-geu.onrender.com`**.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

- **C++ core** (`src/core`): plain C++17 with no Qt types in the headers. A match goes through `SkillIndex` (find candidates) → `MatchingStrategy` (score) → `rankTopK` (min-heap, $O(n \log k)$) → `suggestTeam` (greedy) → `JsonStorage` (save).
- **Scoring**: coverage 40%, complementarity 25%, experience 15%, engagement 10%, diversity 10%. We removed *availability* from Phase I because we had no reliable timetable data. *Engagement* took its 10%: 1.0 if the student has shown interest in the project, otherwise 0.5.
- **Bridge and UI**: `TeamForgeController` connects the core to 51 QML screens and checks permissions in C++. Errors are shown as short messages instead of crashing the app.
- **Storage**: five versioned JSON files with a SHA-256 stamp for the sample data.
- **Web version**: Qt for WebAssembly (Qt 6.12, Emscripten 5.0.5) runs the same C++ and QML in the browser. A Python server handles accounts (salted PBKDF2 hashes, HttpOnly cookies) and stores the same JSON files in PostgreSQL. The browser and server talk over HTTPS with a small JSON REST API. Every save carries a version number, so two people cannot overwrite each other's changes.
- **Build and hosting**: GitHub Actions builds the WebAssembly files on every change. Render builds our Dockerfile and runs the server with a free Render Postgres database.



Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Data structures: `SkillIndex` (hash map), `SkillBST` (AVL tree with prefix search for autocomplete), `SkillGraph` (BFS for related skills), top-*k* ranking with a priority queue, greedy team suggestion.	Mehul (DSA Lead)
Class design: `Student`, `ProjectRequirement`, `Team`, `InterestRequest`; abstract `MatchingStrategy` with weighted and coverage-only versions; `Repository<T>` template; exception hierarchy; input validation.	Siddhant Singh (C++/OOP Lead)
Qt Quick interface: entry screen with 3D logo, three workspaces, Match insights, 25 reusable components; 7 Qt Test programs; Windows test build with a testing checklist; documentation.	Anubhav Bisht (UI/Testing/Docs Lead)
Integration: controller and permission model, interest workflow, JSON storage, session and log, CMake build. Web version: WebAssembly build, sign-in server, PostgreSQL storage, GitHub Actions build and Render deployment.	Mukul Fartiyal (Integration Lead)
Sample data generator with a fixed seed: 560 participants, 8 projects, 549 skills, 86 interest requests.	Mukul Fartiyal, Mehul

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

- **Availability factor:** we had no reliable timetable data, and scheduling made the screens confusing. We replaced it with engagement at the same weight and updated the expected scores in our tests.
- **Keeping three structures in sync:** every profile change has to update the hash index, the AVL tree and the graph. We added rebuild functions and tests that check the rebuilt state matches the step-by-step state.
- **Old sample data:** one build folder kept loading old data. We added a SHA-256 stamp and a backup before replacing files.
- **First web version lost data:** our first Render site was a separate web app that saved to a local file. Render's free plan wipes local files on every restart, so accounts kept disappearing. We replaced it with the real Qt app and moved the data to PostgreSQL.
- **Cloud build error (exit code 126):** a Qt helper script could not run on GitHub's machine. We changed the build to call CMake with Qt's toolchain file directly.
- **App stopped on start in the browser:** Qt's ready-made WebAssembly kit turns off exception catching, but our core uses exceptions for validation. We enabled exception catching (`-fexceptions`) for our own code only, and the app now starts normally.
- **Free hosting limits:** the site sleeps after 15 minutes without visitors, and the free database expires after 30 days. We will open the site before demos and move or upgrade the database before it expires.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
Run the full `ctest` suite on the desktop build and record the results.	Anubhav Bisht
Test the live site end to end with the testing checklist (sign up, edit profile, apply, build a team, reload) and fix any issues.	All members
Compare the weighted strategy with the coverage-only baseline on the 560-profile dataset.	Mehul
Move or upgrade the free database before it expires (about 9 November 2026).	Mukul Fartiyal
Update the repository README (Phase II still says "Upcoming").	Anubhav Bisht
Final demonstration, viva preparation and Phase III report.	All members

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

- A C++/Qt desktop application with Participant, Host and Developer workspaces.
- A matching engine that finds candidates, scores them on five factors, ranks them, suggests a team and shows covered and missing skills.
- Team validation for duplicate members, team size and missing skills.
- Local JSON storage with sample data, backups and session restore.
- A live website that runs the same app in the browser, with sign-in, roles and a shared PostgreSQL database.
- An automatic build on GitHub Actions and deployment on Render.
- Automated tests, a Windows test build, documentation and the DSA/OOP mapping for TCS-302 and TCS-307.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

- **Done:** everything we planned for Phase II in our Phase I report: profiles, projects, candidate search, weighted matching and ranking, team building, skill gap checks, the data structures, the OOP design, local storage and automated tests.
- **Ahead of schedule:** the interest workflow between participants and hosts, the Developer workspace that shows the data structures live, the 560-profile dataset, and the live website with sign-in.
- **Behind schedule:** the recorded results of the full Qt test run, end-to-end testing on the live site, and the README update.
- **Changed from the plan:** availability was replaced by engagement in the score.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

- **Repository:** `https://github.com/anubhav8it/TeamForge` (public), branch `main`, 45 commits as of 10 October 2026.
- **Phase II code:** the `Phase-2/` folder contains `src/core`, `src/app`, `src/web`, `ui/`, `tests/`, `data/`, `assets/`, `web/` (server and sign-in pages), `CMakeLists.txt` and `Dockerfile`.
- **Important commits:** "Add files via upload" (Phase II source); "Create build-web.yml" and "Update build-web.yml" (automatic WebAssembly build); "Create Dockerfile" (Render deployment); "Update CMakeLists.txt" (exception fix for the browser build); "Build TeamForge for WebAssembly" (built files, committed by GitHub Actions).
- **Live site:** `https://teamforge-geu.onrender.com`

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
C++ core compile check (g++, `-Wall -Wextra -Wpedantic`)	Pass	15 core files compiled with 0 warnings.
Unit and integration tests (Qt Test, `ctest`)	To be recorded	7 programs: domain, scoring, DSA, engine, persistence, controller, UI. Expected scores are worked out by hand.
WebAssembly build (GitHub Actions)	Pass	First run failed (exit code 126); fixed. Later runs pass.
Server API tests	Pass	52 checks: sign-up, login, roles, re-login keeps data, conflicts, restart.
Page sync logic	Pass	11 checks: unchanged files skipped, changes saved, conflicts shown.
Live site (10 Oct 2026)	Pass	Health check OK, sign-in works, admin account created, app opens on the entry screen.
End-to-end use on the live site	In progress	Profile edit, reload and re-login with the testing checklist.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

- Desktop application with three workspaces – **Completed**
- Matching engine (search, scoring, ranking, team suggestion, gap check) – **Completed**
- Team validation – **Completed**
- Local storage with sample data and backups – **Completed**
- Live website with sign-in and shared database – **Completed**
- Automatic build and deployment – **Completed**
- Automated tests – **Completed**; recorded results – **In progress**
- Documentation and README update – **In progress**
- Strategy comparison, final demo and viva – **Pending** (Phase III)